



Smart Contract Code Review And Security Analysis Report

CUSTOMER BloomBeans
SCOPE Core Protocol
DATE 02.04.2026

Made in Germany by softstack GmbH

Table of Contents

1. Disclaimer	3
2. About the Project and Company	4
3. Vulnerability & Risk Level	5
4. Auditing Strategy and Techniques Applied	6
5. Metrics	7
5.1 Tested Contract Files	7
5.2 Source Lines & Risk	8
5.3 Capabilities	8
5.4 Dependencies / External Imports	9
5.5 Source Units in Scope	9
6. Scope of Work	11
6.1 Findings Overview	11
6.2 Manual and Automated Vulnerability Test	12
6.2.1 Principal Permanently Locked When All Interest Claimed at Maturity	12
6.2.2 Income Table Mutability Retroactively Breaks Existing Token Accounting	13
6.2.3 Token Transferable While on Loan (Income) -- Loan Obligation Dumping	14
6.2.4 Missing Pause Checks on createLoan and repayLoan (Income)	15
6.2.5 SavingsData Table Mutability Retroactively Breaks Savings Token Accounting	16
6.2.6 Token Transferable While on Loan (Savings) -- Loan Obligation Dumping	17
6.2.7 Missing Pause Checks on createLoan and repayLoan (Savings)	17
6.2.8 Zero dropAmt Allows Permanent Claim Lockout	18
6.2.9 Merkle Proof Uses Non-Standard Single-Hash Leaf Encoding	19
6.2.10 Merkle Root Can Be Changed After Users Start Claiming	19
6.2.11 Centralization Risk -- Single EOA Owner Can Add Contract Addresses	20
6.2.12 getMarker() Returns Wrong Tier at Last Boundary Value	21
6.2.13 Silent Falthrough Returns Default 0 on Unmatched Supply	22
6.2.14 Partial setInterest Update Leaves Stale Marker/Interest Data	22
6.2.15 setInterest Uses onlyOwner Instead of onlyMultiSig	23
6.2.16 Input Validation After External Calls in mintIncome (CEI Violation)	23
6.2.17 mintSavings Allows qty=0	24
7. Executive Summary	25
8. About the Auditor	26
9. Glossary	27

1. Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, investment advice, endorsement of the platform or its products, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

The information presented in this report is confidential and privileged. If you are reading this report, you agree to keep it confidential, not to copy, disclose or disseminate without the agreement of the client. If you are not the intended receptor of this document, remember that any disclosure, copying or dissemination of it is forbidden.

Version / Date	Description
0.1 (23.02.2026)	Layout
0.5 (02.03.2026)	Submitted Findings
0.9 (25.03.2026)	Re-check after mitigation
1.0 (02.04.2026)	Final document
1.1 (28.07.2026)	Codebase re-hash to latest commit; MiCA / MiFID II vocabulary alignment

2. About the Project and Company

Company: BloomBeans Foundation

Address: Calle 53 Este, Marbella Edificio World Trade Center, Panamá 00985-0832

Website: <https://www.bloombeans.io>

GitHub: <https://github.com/BloomBeans>

BloomBeans is a decentralized on-chain financial protocol built on Ethereum. Its native token, **BEAN**, is a utility token under MiCA Regulation (EU) 2023/1114, in which all protocol interactions are denominated. The protocol is designed around a transparent, code-governed emission schedule, with a maximum supply of 21 billion BEAN and an emission rate that decreases as total supply grows.

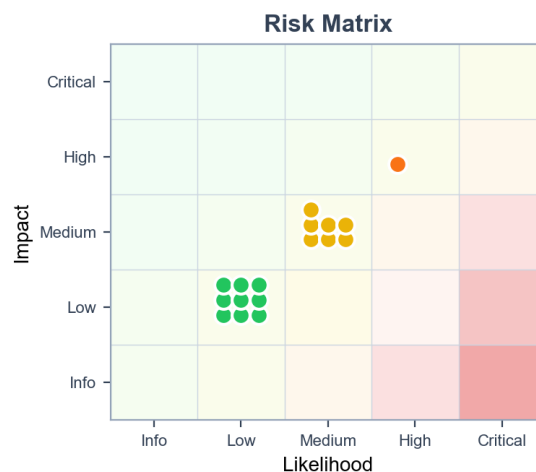
Users interact with the protocol by minting their own ERC-1155 **Smart Trustless Assets (STAs)** — Savings and Income Stream — on-chain BEAN-denominated instruments; BloomBeans does not issue them. Users commit a principal BEAN amount for a chosen period, and the protocol generates mathematically predetermined BEAN rewards at the reward rate set by the **GlobalMarker** system. STAs have no direct equivalent in regulated finance: they can be held to receive scheduled BEAN distributions, used as collateral for BEAN-denominated smart-contract loans, or transferred peer-to-peer. The protocol supports referral rewards, merkle-based airdrops, and an event-driven reward system for community participation.

The audited scope covers the core smart contract suite: the **Bean** ERC-20 token with registered-minter access control, **Income** and **Savings** ERC-1155 STA contracts with BEAN-denominated loan mechanics and migration support, **SavingsData** and metadata libraries, **GlobalMarker** for supply-based reward-rate tiers, **Airdrop** with dual merkle claim systems, **Referral** for tiered reward distribution, **Registry** for contract address management with multisig governance, **Event** for time-based reward allotments, and **Formatters** for on-chain metadata rendering.

3. Vulnerability & Risk Level

Risk represents the probability that a certain source-threat will exploit vulnerability, and the impact of that event on the organization or system. Risk Level is computed based on CVSS version 3.1.

Level	CVSS Score	Vulnerability	Required Action
CRITICAL	9.0 – 10.0	A vulnerability that can disrupt the contract functioning in a number of scenarios, or creates a risk that the contract may be broken.	Immediate action to reduce risk level.
HIGH	7.0 – 8.9	A vulnerability that affects the desired outcome when using a contract, or provides the opportunity to use a contract in an unintended way.	Implementation of corrective actions as soon as possible.
MEDIUM	4.0 – 6.9	A vulnerability that could affect the desired outcome of executing the contract in a specific scenario.	Implementation of corrective actions in a certain period.
LOW	0.1 – 3.9	A vulnerability that does not have a significant impact on possible scenarios for the use of the contract and is probably subjective.	Implementation of certain corrective actions or accepting the risk.
INFORMATIONAL	0.0	A vulnerability that has informational character but is not affecting any of the code.	An observation that does not determine a level of risk.



4. Auditing Strategy and Techniques Applied

Throughout the review process, care was taken to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

The auditing process follows a routine series of steps:

Code review that includes the following:

- Review of the specifications, sources, and instructions provided to softstack to make sure we understand the size, scope, and functionality of the smart contract.
- Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
- Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to softstack describe.

Testing and automated analysis that includes the following:

- Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
- Symbolic execution, which is analysing a program to determine what inputs causes each part of a program to execute.

Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarify, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.

Specific, itemized, actionable recommendations to help you take steps to secure your smart contracts.

5. Metrics

The metrics section gives the reader an overview of the size, quality, flows and capabilities of the codebase.

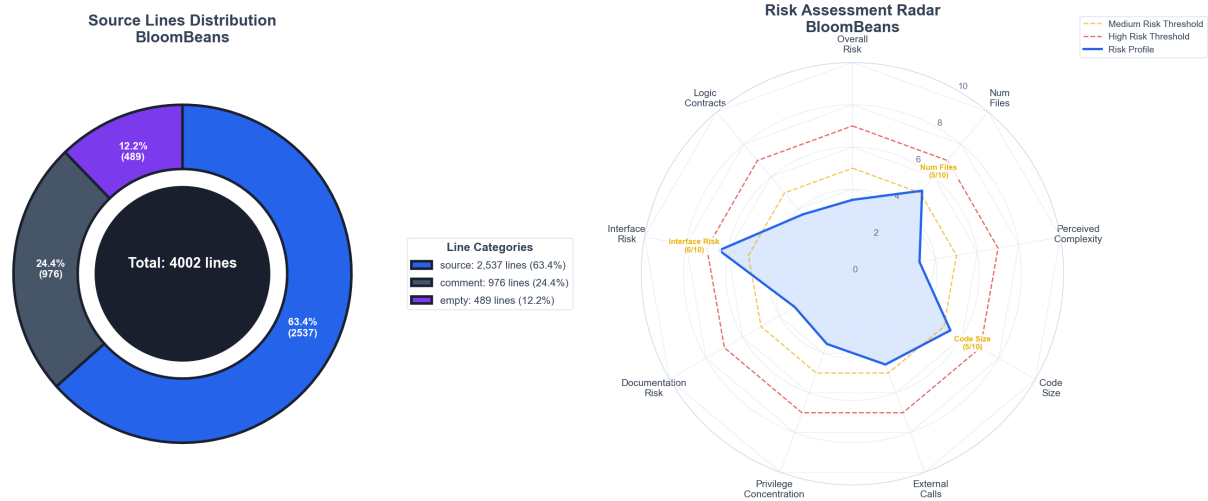
5.1 Tested Contract Files

Source repositories and commits for each contract in scope:

Contract	Source	Commit
BloomBeans Contracts	https://github.com/BloomBeans/bb-contracts	452b42a94243087d2ecb4b8d2e4380f6d144659b

File	Fingerprint (MD5)
./bb-contracts-main/contracts/sta/Income.sol	f83d44218cc0698c49ffbd08329b92b2
./bb-contracts-main/contracts/sta/IncomeData.sol	cb53ad7a1b285e14380097028e9c926e
./bb-contracts-main/contracts/sta/IncomeMetadata.sol	c662feb2f202f6db911179141f7aef93
./bb-contracts-main/contracts/sta/Referral.sol	1d3efcffe4f5c84d3f3de4c27b215f22
./bb-contracts-main/contracts/sta/Savings.sol	f0a28b768559cac25108a90f6652f90c
./bb-contracts-main/contracts/sta/SavingsData.sol	6b211c2e957e5e229d595e2d4607e5d7
./bb-contracts-main/contracts/sta/SavingsMetadata.sol	38d42517e7e1da2347685d1e0aaeac9
./bb-contracts-main/contracts/sta/interface/IIncome.sol	1d9f9020d7ccd00210705caaecf1a2d6
./bb-contracts-main/contracts/sta/interface/IReferral.sol	0b36de877dab5bb39a34b5849ab81b8f
./bb-contracts-main/contracts/sta/interface/ISavings.sol	e11c26b2b47b96c618deeb5abd252049
./bb-contracts-main/contracts/token/Bean.sol	a916414c4c8e65e6189d8bc79e35eabc
./bb-contracts-main/contracts/utils/Airdrop.sol	3b63ffe1e93c282a08e55e61337ab8a5
./bb-contracts-main/contracts/utils/Event.sol	73811d49b5e682c041ceec7a7a1fdbec
./bb-contracts-main/contracts/utils/Formatters.sol	85a0bb42401327490bb52d780c1cc497
./bb-contracts-main/contracts/utils/GlobalMarker.sol	162096f981aebb72dcf063659e417bfb
./bb-contracts-main/contracts/utils/Registry.sol	604449da217bbeb151f26de63f8652f4

5.2 Source Lines & Risk



5.3 Capabilities

Module	Functions	Public	Restricted	Key Roles
Airdrop	10	3	7	owner
Bean	7	6	1	owner
Event	6	2	4	owner
Formatters	3	3	0	-
GlobalMarker	5	3	2	owner
Income	35	28	7	owner
IncomeData	3	2	1	owner
IncomeMetadata	2	2	0	-
Referral	19	14	5	owner
Registry	12	11	1	owner
Savings	33	28	5	owner
SavingsData	2	0	2	owner
SavingsMetadata	1	1	0	-

5.4 Dependencies / External Imports

Import Path	Version
@openzeppelin/contracts/access/Ownable.sol	^5.0.1
@openzeppelin/contracts/token/ERC1155/ERC1155.sol	^5.0.1
@openzeppelin/contracts/token/ERC1155/utils/ERC1155Holder.sol	^5.0.1
@openzeppelin/contracts/token/ERC20/ERC20.sol	^5.0.1
@openzeppelin/contracts/token/ERC20/IERC20.sol	^5.0.1
@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol	^5.0.1
@openzeppelin/contracts/utils/Base64.sol	^5.0.1
@openzeppelin/contracts/utils/ReentrancyGuard.sol	^5.0.1
@openzeppelin/contracts/utils/Strings.sol	^5.0.1
@openzeppelin/contracts/utils/cryptography/MerkleProof.sol	^5.0.1

5.5 Source Units in Scope

File	Instructions	Functions	Lines	nSLOC	Comments
Bean.sol	7	7	171	84	67
Airdrop.sol	6	10	235	135	76
Event.sol	5	6	135	64	52
GlobalMarker.sol	4	5	130	61	53
Formatters.sol	0	4	118	94	4
Registry.sol	10	12	262	124	101
IncomeData.sol	0	3	61	30	23
Savings.sol	21	35	970	588	248
IncomeMetadata.sol	0	8	212	195	4
SavingsData.sol	0	2	35	23	6
Referral.sol	9	21	427	233	142
SavingsMetadata.sol	0	6	157	141	6

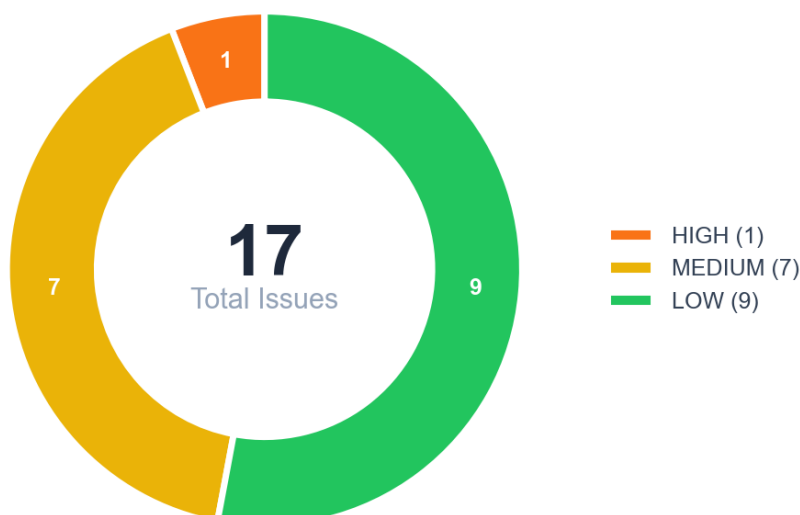
File	Instructions	Functions	Lines	nSLOC	Comments
Income.sol	24	40	994	683	190
ISavings.sol	0	0	37	32	1
IReferral.sol	0	0	13	11	1
IIIncome.sol	0	0	45	39	2
Total	86	159	4002	2537	976

6. Scope of Work

This audit covers the BloomBeans DeFi smart contract suite deployed on Ethereum. The contracts implement the BEAN ERC-20 token, ERC-1155 Smart Trustless Asset (STA) products (Income and Savings), a supply-based reward-rate system (GlobalMarker), merkle-based airdrops, referral rewards, event-driven allotments, and a registry with multisig governance. The audit focuses on validating security, access control, economic logic correctness, loan mechanics, and token lifecycle management.

6.1 Findings Overview

Findings are classified by severity: Critical, High, Medium, Low, and Informational.



No	Title	Severity	Status
6.2.1	Principal Permanently Locked When All Interest Claimed at Maturity	HIGH	FIXED
6.2.2	Income Table Mutability Retroactively Breaks Existing Token Accounting	MEDIUM	FIXED
6.2.3	Token Transferable While on Loan (Income) -- Loan Obligation Dumping	MEDIUM	FIXED
6.2.4	Missing Pause Checks on createLoan and repayLoan (Income)	MEDIUM	FIXED

No	Title	Severity	Status
6.2.5	SavingsData Table Mutability Retroactively Breaks Savings Token Accounting	MEDIUM	FIXED
6.2.6	Token Transferable While on Loan (Savings) -- Loan Obligation Dumping	MEDIUM	FIXED
6.2.7	Missing Pause Checks on createLoan and repayLoan (Savings)	MEDIUM	FIXED
6.2.8	Zero dropAmt Allows Permanent Claim Lockout	MEDIUM	FIXED
6.2.9	Merkle Proof Uses Non-Standard Single-Hash Leaf Encoding	LOW	FIXED
6.2.10	Merkle Root Can Be Changed After Users Start Claiming	LOW	FIXED
6.2.11	Centralization Risk -- Single EOA Owner Can Add Contract Addresses	LOW	FIXED
6.2.12	getMarker() Returns Wrong Tier at Last Boundary Value	LOW	FIXED
6.2.13	Silent Falthrough Returns Default 0 on Unmatched Supply	LOW	FIXED
6.2.14	Partial setInterest Update Leaves Stale Marker/Interest Data	LOW	FIXED
6.2.15	setInterest Uses onlyOwner Instead of onlyMultiSig	LOW	FIXED
6.2.16	Input Validation After External Calls in mintIncome (CEI Violation)	LOW	FIXED
6.2.17	mintSavings Allows qty=0	LOW	FIXED

6.2 Manual and Automated Vulnerability Test

6.2.1 Principal Permanently Locked When All Interest Claimed at Maturity

HIGH

FIXED

DESCRIPTION

withdrawIncome() gates the entire function -- including principal return -- behind a NothingToWithdraw check on withdrawable interest. When a user claims all 12 interest payments at exactly `block.timestamp == maturityDate`, the principal return path (guarded by `strict >`) is skipped. All subsequent calls revert because `getWithdrawableReward()` returns 0. Since `withdrawIncome()` is the only function that returns principal, the deposited principal is permanently locked with no recovery mechanism.

CODE LOCATION

Gate -- Income.sol L300-301:

```
uint256 withdrawableAmount = getWithdrawableReward(_id);
if (withdrawableAmount == 0) revert NothingToWithdraw();
Principal return (unreachable) -- L323-324:
if (block.timestamp > attributes[_id].maturityDate) { // strict >, not >=
token.transfer(msg.sender, attributes[_id].principal);
_burn(msg.sender, _id, 1);
}
```

RECOMMENDATION

Change > to >= for the maturity check. Decouple principal return from the interest withdrawal gate:

```
if (withdrawableAmount > 0) {
// claim interest
}
if (block.timestamp >= attributes[_id].maturityDate) {
token.transfer(msg.sender, attributes[_id].principal);
_burn(msg.sender, _id, 1);
} else if (withdrawableAmount == 0) {
revert NothingToWithdraw();
}
```

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/8bd927e667cd5e087989914cc43658bc781e05c2>

6.2.2 Income Table Mutability Retroactively Breaks Existing Token Accounting

MEDIUM

FIXED

DESCRIPTION

setIncomeTable() can be modified at any time by the owner. Existing Income tokens have earningsRemaining frozen at mint time, but getWithdrawableReward() reads the live incomeTable at withdrawal. If rates increase, withdrawableAmount > earningsRemaining causes arithmetic underflow, permanently locking the token. If rates decrease, users receive less than promised.

CODE LOCATION

Mutable table -- Income.sol L504-513:

```
function setIncomeTable(uint256 _paymentFrequency, uint256[][] memory _multiplier,
uint256 _part) external onlyOwner {
uint256 startIndex = (_part * _multiplier.length) + 1;
for (uint256 i = 0; i < _multiplier.length; i++) {
incomeTable[_paymentFrequency][startIndex + i] = _multiplier[i];
}
```

```

}
}
Frozen at mint -- L233-244:
attributes[system.idCounter].earningsRemaining = totalReward;
Live read at withdrawal -- L665-667:
function getWithdrawableReward(uint256 _id) public view returns (uint256) {
    (uint256 claimableIndex, ) = getIndexes(_id);
    return getMonthlyInterest(_id) * claimableIndex; // reads live incomeTable
}
Underflow site -- L308:
attributes[_id].earningsRemaining -= withdrawableAmount;

```

RECOMMENDATION

Store the per-payment amount in Attributes at mint time instead of reading the live table:

```

attributes[system.idCounter].amountPerInterval = getIncomeOutcome(...);
function getMonthlyInterest(uint256 _id) public view returns (uint256) {
    return attributes[_id].amountPerInterval;
}

```

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/32cc0f026cdf1c3573b841479a53798782327c90>

6.2.3 Token Transferable While on Loan (Income) -- Loan Obligation Dumping

MEDIUM

FIXED

DESCRIPTION

ERC1155 transfer overrides (safeTransferFrom/safeBatchTransferFrom) only check whenNotPaused but never check loan[id].onLoan. A borrower can take a loan (receive 25% of principal as minted BEAN), then transfer the token to a victim. The victim inherits the loan obligation and cannot withdraw until they repay with their own funds.

CODE LOCATION

Transfer override missing loan check -- Income.sol L968:

```

function safeTransferFrom(address from, address to, uint256 id, uint256 amount,
    bytes memory data)
    public virtual override whenNotPaused {
    // No loan[id].onLoan check

```

```
super.safeTransferFrom(from, to, id, amount, data);
}
```

```
function safeTransferFrom(...) public virtual override whenNotPaused {
  if (loan[id].onLoan) revert TokenOnLoan();
  super.safeTransferFrom(from, to, id, amount, data);
}
```

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/394c3f19abab502c8d5b36737feb91467752e8cc>

6.2.4 Missing Pause Checks on createLoan and repayLoan (Income)

MEDIUM

FIXED

DESCRIPTION

emergencyPause is enforced on mintIncome, withdrawIncome, migrateIncome, and transfer overrides, but not on createLoan() (L868) or repayLoan() (L917). During an emergency pause, createLoan still mints new Bean tokens and internally withdraws pending rewards, while repayLoan modifies maturity and interest state.

CODE LOCATION

createLoan missing pause -- Income.sol L868:

```
function createLoan(uint256 _id) external nonReentrant {
  // No whenNotPaused or emergencyPause check
  repayLoan missing pause -- Income.sol L917:
  function repayLoan(uint256 _id) external nonReentrant {
    // No whenNotPaused or emergencyPause check
```

```
- function createLoan(uint256 _id) external nonReentrant {
+ function createLoan(uint256 _id) external nonReentrant whenNotPaused {
- function repayLoan(uint256 _id) external nonReentrant {
+ function repayLoan(uint256 _id) external nonReentrant whenNotPaused {
```

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/f3c0efc3cb68c27d3b9edafefd7e4f82aca8b09d>

6.2.5 SavingsData Table Mutability Retroactively Breaks Savings Token Accounting

MEDIUM

FIXED

DESCRIPTION

SavingsData.savingsTable can be modified at any time via setSavingsTable()/setSavingsTableBatch(). Savings tokens store only lookup keys (productLife, markerIndex) at mint -- not the actual multiplier. getSavingsOutcome() reads the live table at withdrawal. If multipliers increase, the contract may lack sufficient tokens, causing transfer reverts and permanently locking principal + interest. If multipliers decrease, users receive less than promised.

CODE LOCATION

Mutable table -- SavingsData.sol L18-25:

```
function setSavingsTable(uint256[][] memory _multiplier) external onlyOwner {
    for (uint256 i = 0; i < _multiplier.length; i++) {
        savingsTable[i + 1] = _multiplier[i];
    }
}

Live read at withdrawal -- Savings.sol L799-805 getSavingsOutcome():
function getSavingsOutcome(uint256 _period, uint256 _marker, uint256 _amount)
    public view returns (uint256) {
    SavingsData savingsData = SavingsData(registry.getContractAddress("SavingsData"));
    uint256 _multiplier = savingsData.savingsTable(_period, _marker);
    uint256 interest = Math.mulDiv(_amount, _multiplier, 1 ether);
    return interest;
}
```

RECOMMENDATION

Snapshot the computed interest at mint time:

```
attributes[system.idCounter].interestReturn = totalReward;
// At withdrawal: use attributes[_id].interestReturn instead of live table
```

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/65cfc4c932ac0dd4b30501d8594c21265d85bc8a>

6.2.6 Token Transferable While on Loan (Savings) -- Loan Obligation Dumping

MEDIUM

FIXED

DESCRIPTION

Same pattern as H-02. ERC1155 transfer overrides in Savings.sol only check emergencyPause, with no loan[id].onLoan check. A borrower takes a loan, transfers the token to a victim, and keeps the loaned BEAN. The victim holds a locked token and must repay from their own funds.

CODE LOCATION

Transfer override missing loan check -- Savings.sol L910-919 safeTransferFrom:

```
function safeTransferFrom(address from, address to, uint256 id, uint256 amount,
bytes memory data)
public virtual override {
    if (emergencyPause) revert ContractPaused(); // L917
    // No loan[id].onLoan check
    super.safeTransferFrom(from, to, id, amount, data);
}
```

```
function safeTransferFrom(...) public virtual override {
    if (emergencyPause) revert ContractPaused();
    if (loan[id].onLoan) revert TokenOnLoan();
    super.safeTransferFrom(from, to, id, amount, data);
}
```

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/bff73450b363b65dd8352ed11228301eb1d0644c>

6.2.7 Missing Pause Checks on createLoan and repayLoan (Savings)

MEDIUM

FIXED

DESCRIPTION

Same pattern as H-03. createLoan() and repayLoan() in Savings.sol do not check emergencyPause. Four other functions enforce the pause, but loan operations remain callable -- minting new Bean tokens and modifying state during a supposedly frozen period.

CODE LOCATION

createLoan missing pause -- Savings.sol L476 createLoan():

```
function createLoan(uint256 _id) external nonReentrant {
// No emergencyPause check
repayLoan missing pause -- Savings.sol L505 repayLoan():
function repayLoan(uint256 _id) external nonReentrant {
// No emergencyPause check
```

RECOMMENDATION

Add if (emergencyPause) revert ContractPaused(); to both createLoan() and repayLoan().

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/c90f6d80fa517cee5787b194da1cbad5249992dc>

6.2.8 Zero dropAmt Allows Permanent Claim Lockout

MEDIUM

FIXED

DESCRIPTION

If dropAmt is 0 (its uninitialized default), users who call claim() have their claimedDefi/claimedZeely flag set permanently but receive 0 tokens.token.transfer(msg.sender, 0) does not revert. After the admin corrects the drop amount, affected users can never claim again. No resetClaim() function exists.

CODE LOCATION

claim() with zero dropAmt -- Airdrop.sol L50-83 claim():

```
function claim(bytes32[] calldata proof) external {
uint256 system = systemChecker(proof); // L51
require(!claimedDefi[msg.sender], "Airdrop: Already claimed"); // L61
claimedDefi[msg.sender] = true; // L77 - set before checking
amount
token.transfer(msg.sender, systems[system].dropAmt); // L82 - transfers 0 if not
set
}
```

RECOMMENDATION

Also add an admin function to reset claim status for affected users.

```
function claim(bytes32[] calldata proof) external {
uint256 system = systemChecker(proof);
require(systems[system].dropAmt > 0, "Airdrop: dropAmt not set");
```

```
// ...  
}
```

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/4e20f3b7741963d5c9e356b6e341d4b1e06b164e>

6.2.9 Merkle Proof Uses Non-Standard Single-Hash Leaf Encoding

LOW

FIXED

DESCRIPTION

The Merkle leaf is computed as `keccak256(abi.encodePacked(msg.sender))` -- a single hash. OpenZeppelin recommends double-hashing (`keccak256(bytes.concat(keccak256(abi.encode(msg.sender))))`) to prevent second pre-image attacks where an internal Merkle tree node could be accepted as a valid leaf. While practical exploitability is negligible for address-only leaves (20 bytes vs 64-byte internal nodes), it deviates from the standard defense-in-depth pattern.

CODE LOCATION

Airdrop.sol L129-151 -- `systemChecker()`, leaf encoding at L136, L144 (also L70 in `claim()`):

```
bytes32 leaf = keccak256(abi.encodePacked(msg.sender));
```

RECOMMENDATION

Use the double-hash pattern:

```
bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(msg.sender))));
```

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/7162eb5e33314c08f80099d0e1e23e82cb46bbf1>

6.2.10 Merkle Root Can Be Changed After Users Start Claiming

LOW

FIXED

DESCRIPTION

changeMerkleRoot() is callable by the owner at any time with no timelock, event, or delay -- even while the airdrop is active. Users with pending claims in the mempool may have their proofs invalidated. Claimed mappings are not reset, so users who claimed under the old root are permanently locked out of the new root.

CODE LOCATION

Airdrop.sol L191-196 -- changeMerkleRoot():

```
function changeMerkleRoot(uint256 _index, bytes32 _root) external onlyOwner {
    systems[_index].merkleRoot = _root;
}
```

RECOMMENDATION

Add a timelock or delay mechanism, emit events on root changes, and consider adding a resetClaimed() admin function for migrations.

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/e5af3dfc5ccab631134bf6b63c70a43b9d6a6aac>

6.2.11 Centralization Risk -- Single EOA Owner Can Add Contract Addresses

LOW

FIXED

DESCRIPTION

Before setRegistry() is called and Registry.initializeMultiSig() completes, the onlyMultiSig modifier falls back to owner(). During this window, a single EOA controls setRegisteredContracts() (register arbitrary minting contracts), unregisterContract(), and withdrawAccidentalBeans(). Additionally, setRegistry() is onlyOwner with no restriction on re-calling it -- the owner can deploy a malicious Registry where getMultiSig() returns address(0) to reclaim control after multisig setup.

CODE LOCATION

Bean.sol L48-64 onlyMultiSig modifier and L122-125 setRegistry():

```
modifier onlyMultiSig() {
    if (address(registry) != address(0)) {
        require(msg.sender == registry.getMultiSig(), "Not MultiSig");
    } else {
        require(msg.sender == owner(), "Not Owner");
    }
    _;
}
```

```
function setRegistry(address _registry) external onlyOwner {
    registry = IRegistry(_registry);
}
```

RECOMMENDATION

Require registry in constructor, add one-time-set guard to setRegistry(), renounce ownership after setup.

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/5e98eec2645a5f311730734c621a2a7b8f9f492d>

6.2.12 getMarker() Returns Wrong Tier at Last Boundary Value

LOW

FIXED

DESCRIPTION

When totalSupply equals exactly markers[markers.length - 1] (the last marker value), getMarker() returns 0 (first tier / highest interest) instead of the last tier. The first condition uses strict > and the loop uses strict <, so the exact boundary value falls through to an implicit return of 0.

CODE LOCATION

GlobalMarker.sol L81-100 -- getMarker(), boundary check at L87, loop at L90-98:

```
function getMarker() public view returns (uint256) {
    uint256 totalSupply = bean.totalSupply();
    if (totalSupply > markers[markers.length - 1]) return markers.length - 1;
    for (uint256 i = 0; i < markers.length; i++) {
        if (totalSupply < markers[i]) return i;
    }
    // implicit return 0
}
```

RECOMMENDATION

Change the first condition from > to >=:

```
if (totalSupply >= markers[markers.length - 1]) return markers.length - 1;
```

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/b9b9cabf49ed4d107d99d2ac76a1015a6fd6e573>

6.2.13 Silent Fallthrough Returns Default 0 on Unmatched Supply

LOW

FIXED

DESCRIPTION

getMarker() has a code path where the function completes without an explicit return statement, producing an implicit return of 0 (tier 0 = highest interest). This silent fallthrough masks logic errors and should revert instead of silently returning a default value.

CODE LOCATION

GlobalMarker.sol L81-100 -- getMarker() end of function, implicit return after L98:

```
for (uint256 i = 0; i < markers.length; i++) {  
  if (totalSupply < markers[i]) return i;  
}  
// falls through here with no return/revert
```

RECOMMENDATION

Add explicit revert after the loop:

```
revert("GlobalMarker: No matching tier");
```

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/b9b9cabf49ed4d107d99d2ac76a1015a6fd6e573>

6.2.14 Partial setInterest Update Leaves Stale Marker/Interest Data

LOW

FIXED

DESCRIPTION

The constructor allocates 41-element arrays. setInterest() writes index-by-index without clearing remaining slots. If fewer than 41 entries are provided, stale values persist. If only 10 markers are set (indices 0-9), indices 10-40 remain 0, causing getMarker() to short-circuit on any positive totalSupply > markers[40] (which is 0), returning tier 40 with 0% interest.

CODE LOCATION

GlobalMarker.sol L48-61 -- setInterest(), for loop at L57-60:

```
function setInterest(uint256[] calldata _marker, uint256[] calldata _interest)  
external onlyOwner {  
  for (uint256 i = 0; i < _marker.length; i++) {
```

```
markers[i] = _marker[i];
interest[i] = _interest[i];
}
}
```

RECOMMENDATION

Replace index-by-index write with full array replacement:

```
markers = _marker;
interest = _interest;
```

6.2.15 setInterest Uses onlyOwner Instead of onlyMultiSig

LOW

FIXED

DESCRIPTION

setInterest() -- the protocol's interest rate oracle configuration -- uses onlyOwner instead of the onlyMultiSig pattern used for lesser functions elsewhere. A compromised owner key can unilaterally set all interest rates to 0% or maximum.

CODE LOCATION

GlobalMarker.sol L48 -- setInterest():

```
function setInterest(uint256[] calldata _marker, uint256[] calldata _interest)
external onlyOwner {
```

RECOMMENDATION

Change to onlyMultiSig or transfer ownership to the MultiSig wallet.

6.2.16 Input Validation After External Calls in mintIncome (CEI Violation)

LOW

FIXED

DESCRIPTION

mintIncome validates _paymentFrequency and _period inside the minting loop (lines 213-214), after Referral.addReferrer(), Referral.eligibleForReward(), Bean.mint(), and IERC20.transferFrom() have already executed. With _qty = 0, validation never executes at all (loop body is skipped).

CODE LOCATION

Income.sol L167-254 -- mintIncome(), addReferrer at L181, validation at L214-215:

```
function mintIncome(..., uint256 _qty, ...) external {
  referral.addReferrer(...); // external call first
  ...
  for (uint256 i; i < _qty; i++) {
    if (_paymentFrequency < system.minPaymentFrequency || _paymentFrequency >
        system.maxPaymentFrequency) revert InvalidPaymentFrequency();
    if (_period < system.minPeriod || _period > system.maxPeriod) revert
        InvalidPeriod();
    // ... minting logic
  }
}
```

RECOMMENDATION

Move all input validation to the top of mintIncome, before any external calls.

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/35bf9dd6d0ff1625735a5273518cbf6af80d2ef4>

6.2.17 mintSavings Allows qty=0

LOW

FIXED

DESCRIPTION

Same pattern as M-14. Calling mintSavings with _qty = 0 passes the max check. The referral registration executes before the loop, but no tokens are minted. Enables free referral chain gaming without economic commitment.

CODE LOCATION

Savings.sol L153-260 -- mintSavings(), qty check at L160, addReferrer at L179:

```
function mintSavings(uint256 _amount, uint256 _period, uint256 _qty, address
_referrer) external {
  if (_qty > MAX_QTY) revert MaxQtyExceeded(); // 0 passes
  referral.addReferrer(msg.sender, _referrer); // executes regardless
  for (uint256 i; i < _qty; i++) { // loop skipped
```

```
  if (_qty == 0 || _qty > MAX_QTY) revert MaxQtyExceeded();
```

MITIGATION

<https://github.com/BloomBeans/bb-contracts/commit/7656ce3d64d3092476655ad3f92d9e6e4f247590>

7. Executive Summary

Two independent softstack experts performed an unbiased and isolated audit of the smart contract provided by the BloomBeans team. The main objective of the audit was to verify the security and functionality claims of the smart contract. The audit process involved a thorough manual code review and automated security testing.

Overall, the audit identified a total of 17 issues, classified as follows:

- No critical issues were found
- 1 high severity issue was found
- 7 medium severity issues were found
- 9 low severity issues were found
- No informational issues were found

The BloomBeans team has successfully addressed all identified issues from the audit. All vulnerabilities have been mitigated based on the recommendations provided in the report. A follow-up review confirms that the fixes have been implemented effectively, ensuring the security and functionality of the smart contract.

8. About the Auditor

Established in 2017 under the name Chainsulting, and rebranded as softstack GmbH in 2023, softstack has been a trusted name in Web3 Security space. Within the rapidly growing Web3 industry, softstack provides a comprehensive range of offerings that include software development, cybersecurity, and consulting services. Softstack's competency extends across the security landscape of prominent blockchains like Solana, Tezos, TON, Ethereum and Polygon. The company is widely recognized for conducting thorough code audits aimed at mitigating risk and promoting transparency.

The firm's proficiency lies particularly in assessing and fortifying smart contracts of leading DeFi projects, a testament to their commitment to maintaining the integrity of these innovative financial platforms. To date, softstack plays a crucial role in safeguarding over \$100 billion worth of user funds in various DeFi protocols.

Underpinned by a team of industry veterans possessing robust technical knowledge in the Web3 domain, softstack offers industry-leading smart contract audit services. Committed to evolving with their clients' ever-changing business needs, softstack's approach is as dynamic and innovative as the industry it serves.

Check our website for further information: <https://softstack.io>

9. Glossary

Term	Definition
BEAN	Native utility token of the BloomBeans protocol (ERC-20)
CEI	Checks-Effects-Interactions pattern for reentrancy prevention
CVSS	Common Vulnerability Scoring System
DeFi	Decentralized Finance
EOA	Externally Owned Account (user wallet)
ERC-1155	Ethereum multi-token standard supporting fungible and non-fungible tokens
ERC-20	Ethereum token standard for fungible tokens
OZ	OpenZeppelin, widely-used Solidity library for secure smart contracts
STA	Smart Trustless Asset — on-chain assets with economic functionality (ERC-1155)